

Zero-Shot Reactive Obstacle Avoidance for Generative Robot Policies

Weihsang Guo and Lydia E. Kavraki

Abstract—We propose **NUDGE** (**N**udge **U**pdate via **D**ifferentiable **G**Eometry), a training-free obstacle-avoidance procedure that can be incorporated in any robot policy based on diffusion or flow matching, including diffusion policies and vision-language-action models. Our work injects gradients from a signed distance field, a function returning each point’s distance to the nearest obstacle, into the policy at inference time to steer it away from obstacles. It supports any common action parameterization, from absolute or relative joint poses to end-effector poses, through a differentiable joint-trajectory decoder. Experiments show that NUDGE preserves the policy’s task distribution and runs reactively in real time.

I. INTRODUCTION

Modern robotics policies [1–3] learn closed-loop visuomotor control and handle implicit task constraints that are hard to specify analytically but are easy to demonstrate, such as keeping a cup upright while pouring, maintaining contact while wiping, or holding a grasp through transport. However, policies are trained on obstacle-free demonstrations. When deployed in environments with novel obstacles, the robot may collide with them [4]. This problem raises safety concerns.

We consider inference-time obstacle avoidance for a pre-trained action-chunk policy that was trained only on obstacle-free demonstrations. We propose NUDGE (Nudge Update via Differentiable GEometry), a training-free procedure that adds obstacle avoidance without overriding the learned task distribution. NUDGE requires only that the policy exposes a denoised action chunk at each sampling step, a property shared by diffusion, flow-matching, and vision-language-action samplers. At each step, NUDGE evaluates the predicted action chunk using a signed distance field (SDF), which maps each workspace point to its signed distance from the nearest obstacle surface, and nudges the chunk against the cost gradient before the sampler re-encodes it to the next noise level. Subsequent sampling steps absorb the nudge, so the policy’s output stays close to the demonstration manifold while avoiding obstacles encoded in the SDF.

In this paper, we contribute 1) A training-free, model-agnostic guidance procedure for obstacle avoidance that works on any action-chunk policy based on diffusion or flow matching, covering diffusion policies and vision-language-action models. 2) Empirical evidence on two simulation benchmarks that NUDGE preserves the policy’s learned task distribution while avoiding obstacles. 3) Deployment on a

fine-tuned π_0 VLA driving a UR5 against dynamic obstacles, with NUDGE adding only 2.5 ms per action chunk.

II. RELATED WORK

We distinguish two largely separate threads of work that use generative models for robot motion. *Motion planners* take a start and goal configuration and output an entire collision-free trajectory in a single pass, with obstacle geometry encoded as a cost or constraint [5–13]. *Task policies* instead learn closed-loop visuomotor behavior from demonstrations and produce short action chunks conditioned on observations. Task policies encode task-level constraints such as grasp orientation or surface contact implicitly through training data, without a dedicated geometry-aware collision cost. *NUDGE targets the task policies category*, adding inference-time collision avoidance to pretrained task policies without disturbing the learned task distribution.

A. Generative Task Policies

Task policies map current observations to short action chunks, learning task behavior from demonstrations. Action chunking was introduced by ACT [14], and Diffusion Policy [1] formulates it as conditional denoising over chunks. $\pi_{0.5}$ [3] and RDT [15] extend the same idea to vision-language-action models by pairing a pretrained VLM with a flow-matching or diffusion action head. NUDGE also supports other newer flow-matching policies such as Streaming Flow Policy [16], Action-to-Action Flow Matching [17], and VITA [18]. Both classes learn task-level constraints (distribution) implicitly through demonstration data, such as keeping a cup upright or maintaining contact with a surface. Because obstacle avoidance is not part of their training signal, the policy can cause collisions between the robot and the environment when obstacles are present. NUDGE adds the missing obstacle awareness at inference without retraining or modifying the policy. We note that autoregressive based policies such as OpenVLA [19] and RT-2 [20] fall outside NUDGE scope. They generate actions one token at a time, with no intermediate estimate of the full chunk for guidance to act on.

B. Guided Diffusion and Obstacle Avoidance

Guidance steers pretrained diffusion models toward auxiliary objectives without retraining, by adding a cost gradient to the learned score [21–23] or by hard-projecting onto a constraint set [24]. NUDGE follows Diffusion Posterior Sampling [23] in evaluating the cost on the predicted action chunk rather than the noisy intermediate, and propagates the

The authors are with the Department of Computer Science, Rice University, Houston, TX, USA. {wg25, kavraki}@rice.edu. LEK is also affiliated with the Ken Kennedy Institute at Rice University.

This work was supported in part by Enrich funds at Rice University.

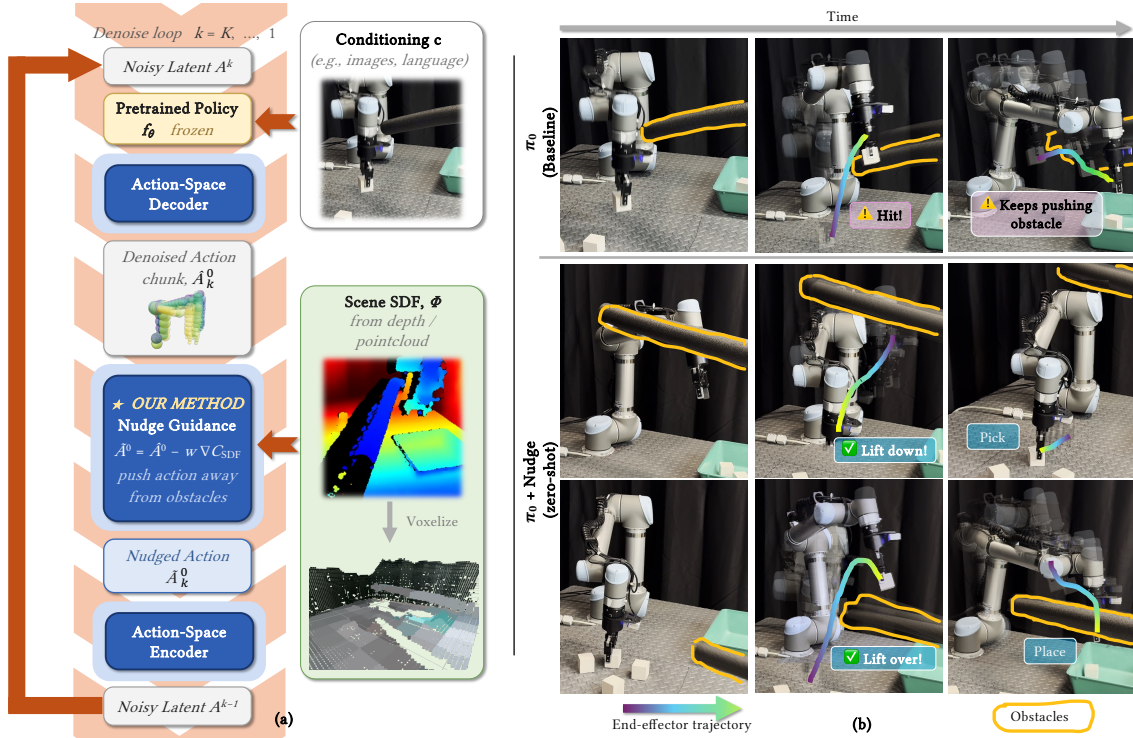


Fig. 1. **NUDGE provides zero-shot collision avoidance for generative robot policies at inference time.** (a) From a live point cloud, NUDGE computes a signed distance field in *real time* to react to scene changes, injects its gradient into the policy’s denoising loop, and steers the predicted action chunk away from obstacles. (b) We fine-tuned a π_0 policy for a table-cleaning task, where the policy learns to model the contact and task distribution from demonstrations. **Top:** the unguided baseline collides with the pool noodle. **Middle & Bottom:** NUDGE avoids the obstacle at the extra cost of only 2.5 ms per action chunk.

gradient analytically through differentiable signed distance field and robot kinematics.

Within robotics research, DynaGuide [25], VLS [26], ACG [27], and Lan-o3dp [28] apply similar inference-time guidance using learned dynamics, VLM-synthesized rewards, or object-centric representations, but none target whole-body collision avoidance. RAIL [4] is a reachability-based safety filter that runs on top of a pretrained Diffusion Policy: each predicted chunk is checked for collision, and an unsafe chunk is replaced with a precomputed safe backup. NUDGE instead edits the chunk in place. Classical filters such as CBFs [29], reachability analysis [30], and residual policies [31] apply orthogonally to our inference-time setting, as does safe-RL exemplified by CPO [32]. Dedicated reactive planners such as cuRoboV2 [33] SDFs [34], STORM [35], Neural MP [36], and Deep Reactive Policy [9] produce obstacle-aware trajectories but lack the task semantics encoded by demonstration training. Concurrent with our work, OmniGuide [37] also guides generalist policies at inference time that guides a sparse set of Cartesian probe points, primarily the end-effector. NUDGE instead focuses on whole-body obstacle avoidance, supporting joint- and end-effector-space actions via a differentiable decoder.

III. METHOD

We consider inference-time obstacle avoidance for a pre-trained action-chunk policy that was trained only on obstacle-

free demonstrations. The pretrained policy may be any action-chunk policy based on diffusion or flow matching, including diffusion policies and vision-language-action models. At deployment, we are given the current robot state q_t , the conditioning input c that the policy expects, and a signed distance field Φ of the workspace constructed in *real time* to react to scene changes. The goal is to produce action chunks that complete the task while minimizing the collision cost induced by Φ , without retraining the policy and without pushing its outputs off the learned task distribution.

At a high level, NUDGE treats each intermediate denoised action chunk as a candidate short-horizon robot motion. A differentiable decoder maps the chunk from the policy’s action representation to a sequence of future joint configurations. NUDGE then evaluates whole-body obstacle clearance along this trajectory and differentiates the resulting collision cost with respect to the action chunk. The chunk is shifted along the negative cost gradient and passed back to the sampler. Thus, subsequent steps incorporate the correction and retain the task behavior encoded by the pretrained policy.

A. Iterative Action-Chunk Generative Policies

Modern visuomotor policies [1, 3] model $p(\mathbf{A} | c)$ over action chunks $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_H] \in \mathbb{R}^{H \times d}$ of joint- or end-effector (EEF)-space actions. Conditioning c may combine images, language, and proprioception. Sampling refines Gaussian noise $\mathbf{A}^K \sim \mathcal{N}(0, I)$ through $\mathbf{A}^K \rightarrow \dots \rightarrow \mathbf{A}^0$

over K steps. At step k , the network $f_\theta(\mathbf{A}^k, k, \mathbf{c})$ predicts noise, velocity, or a clean target; a formulation-specific decoder h converts it into a clean-chunk estimate $\hat{\mathbf{A}}_k^0$. NUDGE requires this intermediate estimate: $\mathbf{A}^k \mapsto \hat{\mathbf{A}}_k^0 = h(\mathbf{A}^k, f_\theta(\mathbf{A}^k, k, \mathbf{c})) \mapsto \mathbf{A}^{k-1}$, where guidance modifies $\hat{\mathbf{A}}_k^0$ between clean-chunk prediction and re-encoding to the next noise or time level.

Example III.1 (DDIM diffusion policy). For a DDPM-trained diffusion policy [38] sampled with DDIM [39], $f_\theta = \epsilon_\theta$ is a noise predictor and the denoised action chunk is $\hat{\mathbf{A}}_k^0 = \frac{1}{\sqrt{\alpha_k}}(\mathbf{A}^k - \sqrt{1 - \alpha_k} \epsilon_\theta(\mathbf{A}^k, k, \mathbf{c}))$, followed by re-diffusion $\mathbf{A}^{k-1} = \sqrt{\alpha_{k-1}} \hat{\mathbf{A}}_k^0 + \sqrt{1 - \alpha_{k-1}} \epsilon_\theta(\mathbf{A}^k, k, \mathbf{c})$.

Example III.2 (Flow-matching policy). For a flow-matching policy [3] along the linear path $\mathbf{A}^k = (1 - t_k) \mathbf{A}^0 + t_k \epsilon$ with $t_k \in [0, 1]$, $f_\theta = v_\theta$ is a velocity field and the denoised action chunk is $\hat{\mathbf{A}}_k^0 = \mathbf{A}^k - t_k v_\theta(\mathbf{A}^k, t_k, \mathbf{c})$, followed by an Euler integration step toward t_{k-1} .

Remark III.3 (Generality). The interface is independent of network architecture and conditioning modality, and supports absolute or relative joint and EEF poses.

B. Action-Space Decoder

To remain agnostic to the policy's action representation, we evaluate all downstream costs on a predicted joint trajectory $q_{t+1:t+H}$ obtained from the action chunk through a differentiable decoder. Let $q_t \in \mathbb{R}^{n_q}$ be the current joint state for an n_q -DOF arm, and let $\mathcal{D}$ be a differentiable decoder

$$q_{t+i} = \mathcal{D}_i(\mathbf{A}; q_t), \quad i = 1, \dots, H, \quad (1)$$

that maps the action chunk and the current joint state to the i -th future joint configuration. Table I lists $\mathcal{D}$ for the action representations in common use, covering both joint-space and end-effector-space variants.

TABLE I
ACTION-SPACE DECODERS

Action representation	Notation	Decoder $q_{t+i} = \mathcal{D}_i(\mathbf{A}; q_t)$
Joint pose	$\mathbf{a}_i = q_i^{\text{tgt}}$	$q_{t+i} = \mathbf{a}_i$
Joint delta	$\mathbf{a}_i = \Delta q_i$	$q_{t+i} = q_t + \sum_{j=1}^i \mathbf{a}_j$
EEF pose	$\mathbf{a}_i = T_i^{\text{ee}}$	$q_{t+i} = \text{IK}(\mathbf{a}_i; q_{t+i-1})$
EEF delta	$\mathbf{a}_i = \Delta T_i^{\text{ee}}$	$q_{t+i} = \text{IK}(T_i^{\text{ee}} \cdot \prod_{j=1}^i \mathbf{a}_j; q_{t+i-1})$

C. SDF-Guided Denoising

We adopt a classifier-guidance [21] formulation of obstacle avoidance. The cost is evaluated on the predicted joint trajectory $q_{t+i} = \mathcal{D}_i(\mathbf{A}; q_t)$, $i = 1, \dots, H$, with $\mathcal{D}$ drawn from Table I to match the policy's action representation. We assume access to an SDF updated from the latest scene observation, $\Phi: \mathbb{R}^3 \rightarrow \mathbb{R}$ of the obstacles in the workspace, with $\Phi(p) > 0$ in free space, $\Phi(p) < 0$ inside an obstacle, and $|\Phi(p)|$ equal to the Euclidean distance from p to the nearest obstacle surface. For each predicted configuration we query Φ at points on the robot's collision geometry. Following common practice in SDF-based motion planning [40,41],

let $\mathcal{S}_l = \{(c^{l,j}, r^{l,j})\}_{j=1}^{N_l}$ be the sphere decomposition of link l (positions and radii in link frame) and let $T_l(q) \in \text{SE}(3)$ be the forward-kinematics transform of link l at configuration q . We define the per-chunk collision cost as a one-sided hinge over all robot spheres at all chunk steps:

$$\mathcal{C}_{\text{SDF}}(\hat{\mathbf{A}}^0) = \sum_{i=1}^H \sum_l \sum_{j=1}^{N_l} [d_{\text{safe}} - \Phi(T_l(q_{t+i}) c^{l,j}) + r^{l,j}]_+^2, \quad (2)$$

where $[\cdot]_+ = \max(0, \cdot)$ and $d_{\text{safe}} \geq 0$ is a safety margin. The cost is zero whenever every sphere clears every obstacle by at least d_{safe} . Held objects enter the same sum through collision spheres attached to the gripper. We evaluate the discrete configurations in the action chunk; the objective does not check continuous swept volumes between them.

The gradient of $\mathcal{C}_{\text{SDF}}$ with respect to joint configurations is computed by automatic differentiation through the forward kinematics: $\frac{\partial \mathcal{C}_{\text{SDF}}}{\partial q_{t+i}} = \sum_{l,j} 2[d_{\text{safe}} - \Phi + r^{l,j}]_+ (-\nabla_p \Phi)^\top J^{l,j}(q_{t+i})$, where $J^{l,j}(q) = \partial(T_l(q) c^{l,j}) / \partial q$ is the standard manipulator Jacobian of sphere j on link l . Mapping the gradient back to action space is a single chain-rule application through $\mathcal{D}$:

$$\frac{\partial \mathcal{C}_{\text{SDF}}}{\partial \mathbf{a}_m} = \sum_{i=1}^H \left(\frac{\partial \mathcal{D}_i}{\partial \mathbf{a}_m} \right)^\top \frac{\partial \mathcal{C}_{\text{SDF}}}{\partial q_{t+i}}, \quad m = 1, \dots, H. \quad (3)$$

Example III.4 (Joint-delta decoder). For the joint-delta row of Table I, i.e., $q_{t+i} = q_t + \sum_{j=1}^i \Delta q_j$, the Jacobian $\partial \mathcal{D}_i / \partial \Delta q_m$ is the identity when $m \leq i$ and zero otherwise, so Eq. (3) collapses to a reverse cumulative sum: $\frac{\partial \mathcal{C}_{\text{SDF}}}{\partial \Delta q_m} = \sum_{i=m}^H \frac{\partial \mathcal{C}_{\text{SDF}}}{\partial q_{t+i}}$, for $m = 1, \dots, H$.

D. Guided Sampling Step

At each sampling step we inject the SDF gradient by nudging the denoised action chunk $\hat{\mathbf{A}}_k^0$ before the sampler re-encodes it to the next level:

$$\tilde{\mathbf{A}}_k^0 = \hat{\mathbf{A}}_k^0 - w_k \nabla_{\hat{\mathbf{A}}^0} \mathcal{C}_{\text{SDF}}(\hat{\mathbf{A}}_k^0), \quad (4)$$

where $w_k \geq 0$ sets the guidance strength; zero disables guidance. The sampler re-encodes $\tilde{\mathbf{A}}_k^0$ instead of $\hat{\mathbf{A}}_k^0$ to produce $\mathbf{A}^{k-1}$, as illustrated below.

Example III.5 (Guided DDIM step). For a DDIM sampler, $\tilde{\mathbf{A}}_k^0$ replaces $\hat{\mathbf{A}}_k^0$ in the re-diffusion update: $\mathbf{A}^{k-1} = \sqrt{\alpha_{k-1}} \tilde{\mathbf{A}}_k^0 + \sqrt{1 - \alpha_{k-1}} \epsilon_\theta(\mathbf{A}^k, k, \mathbf{c})$.

Example III.6 (Guided flow-matching step). For a flow-matching sampler with the linear path, the Euler step toward t_{k-1} can be written as a linear interpolation between the current latent and the denoised action chunk; replacing $\hat{\mathbf{A}}_k^0$ with $\tilde{\mathbf{A}}_k^0$ gives the guided update $\mathbf{A}^{k-1} = \frac{t_{k-1}}{t_k} \mathbf{A}^k + \frac{t_k - t_{k-1}}{t_k} \tilde{\mathbf{A}}_k^0$.

At deployment, each control step executes the first $H_{\text{exec}} \leq H$ predicted actions and re-runs the sampler against the latest world-frame SDF Φ . Algorithm 1 gives the DDIM procedure. Flow matching replaces the update with Example III.6. Raw scales depend on the policy's joint- or EEF-action coordinates. For example, LIBERO [42]'s

action-to-displacement conversion introduces an approximately 0.0085 m factor into the decoder Jacobian, so a large numerical w_k need not imply a large physical correction. We can reparameterize the scale as

$$\rho_k = \frac{w_k \kappa}{1 + w_k \kappa} \in [0, 1), \quad w_k = \frac{\rho_k}{\kappa(1 - \rho_k)}, \quad (5)$$

where $\kappa > 0$ is the median nonzero ℓ_∞ gradient norm on a fixed calibration set in normalized action coordinates. It measures the typical gradient magnitude, allowing search over bounded ρ_k . This invertible mapping preserves the update and subsequent clipping.

Algorithm 1 Guided Action Chunk Generation

Require: Joints q_t , conditioning $\mathbf{c}$, policy f_θ , denoising decoder h , action decoder $\mathcal{D}$, SDF Φ , horizon H , steps K , scales $\{\rho_k\}_{k=1}^K$, calibration factor $\kappa > 0$

- 1: Sample $\mathbf{A}^K \sim \mathcal{N}(0, I)$
- 2: **for** $k = K, K - 1, \dots, 1$ **do**
- 3: $w_k \leftarrow \rho_k / [\kappa(1 - \rho_k)]$ // Eq. (5)
- 4: $\hat{\mathbf{A}}_k^0 \leftarrow h(\mathbf{A}^k, f_\theta(\mathbf{A}^k, k, \mathbf{c}))$ // decode, Sec. III-A
- 5: $\mathbf{g} \leftarrow \nabla_{\hat{\mathbf{A}}_k^0} \mathcal{C}_{\text{SDF}}(\hat{\mathbf{A}}_k^0)$ // Eqs. (1), (2), (3)
- 6: $\tilde{\mathbf{A}}_k^0 \leftarrow \hat{\mathbf{A}}_k^0 - w_k \mathbf{g}$ // Eq. (4)
- 7: $\mathbf{A}^{k-1} \leftarrow \text{ReEncode}_k(\mathbf{A}^k, \tilde{\mathbf{A}}_k^0)$ // Ex. III.5/III.6
- 8: **end for**
- 9: **return** $\mathbf{A}^0$

IV. EXPERIMENTS

Our experiments answer three questions:

- *Does inference-time SDF guidance prevent collisions while preserving the learned task manifold?* Section IV-B answers this on a controllable sphere-constraint task whose manifold is explicit and measurable.
- *Does NUDGE generalize to policies whose task constraints are only implicit in demonstration data, without hurting task performance?* Section IV-C answers this by inserting unseen obstacles into LIBERO-Object scenes.
- *Does NUDGE work on a real-robot VLA, and is SDF guidance a real-time bottleneck?* Section IV-D answers this by deploying NUDGE on a fine-tuned π_0 driving a UR5 against dynamic obstacles, with an end-to-end timing breakdown.

A. Baselines and Robot Collision Geometry

We include RAIL [4] and OmniGuide [37] as our baselines. RAIL enforces collision avoidance as a hard constraint through post-generation verification. After the policy has generated a complete action chunk, RAIL checks the chunk for collisions. If it is unsafe, RAIL replaces it with a safe stopping trajectory instead of guiding or modifying the action chunk. By rejecting rather than correcting unsafe trajectories, RAIL may sacrifice task success to enforce hard safety.

OmniGuide [37], concurrent work only on arXiv, guides pretrained policies at inference time, but evaluates collision costs using only a sparse set of Cartesian probe points around the end effector and wrist, as shown in Fig. 2. OmniGuide

constructs the environment with VGGT [43]. Consequently, it does not explicitly account for collisions involving the remaining robot links. By contrast, we approximate robot links with spheres for *whole-body* collision checking based on cuRoboV2 [33]. Each SDF query, minus the sphere radius, gives the clearance used in $\mathcal{C}_{\text{SDF}}$.

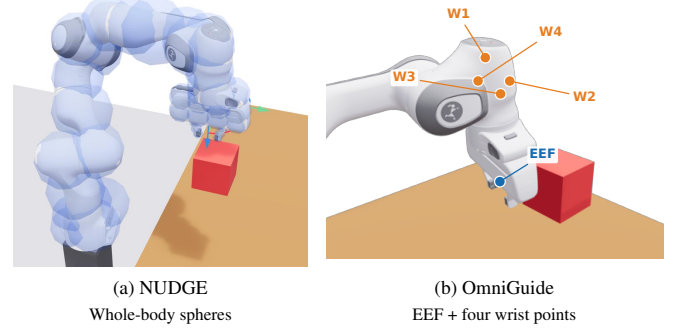


Fig. 2. Collision-query geometry: NUDGE’s whole-body spheres versus OmniGuide’s EEf point and four wrist probe.

B. Sphere Constraint

The sphere-constrained task isolates manifold preservation using an explicit, quantifiable manifold. The sphere serves as a proxy for harder-to-measure manifolds in grasping, pushing, or balancing, and we measure manifold violation as $e = \|\mathbf{p}_{\text{eef}} - \mathbf{c}\| - r$, the distance from the end-effector to the sphere surface (Fig. 3). This setup enables us to quantifiably measure how close the algorithm maintains the task manifold constraint.

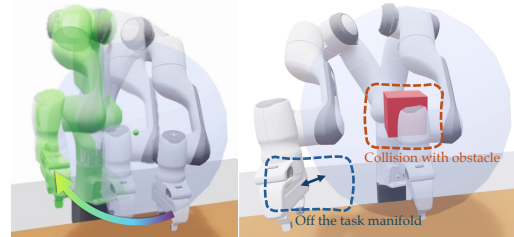


Fig. 3. **Left.** The robot trajectory from start to goal with end-effort constrained on the sphere manifold. **Right.** Failure modes on the sphere constraint task: collides with the obstacle and pushes the end-effector off the learned task manifold.

A Vanilla diffusion policy [1] is trained on 21,760 obstacle-free trajectories of a 7-DOF Franka Panda on a sphere of radius $r = 0.25$ m, conditioned on current joints and goal pose. Action chunks are $H = 16$ joint deltas sampled with DDIM over $K = 10$ steps. Test boxes have side lengths 6–12 cm and centers within ± 8 cm of the sphere center; start and goal are at least 60° apart on the surface. We evaluate 200 episodes with identical starts, goals, and obstacles across methods. We compare against diffusion policy (Vanilla), Post-Projection, which applies joint-space collision correction after sampling, and RAIL [4], which substitutes an ARMTD [44] failsafe whenever the predicted chunk could collide. NUDGE uses

TABLE II
SPHERE-MANIFOLD RESULTS

Method	Success $\uparrow$	Goal	Collision		Manifold	
		Reach $\uparrow$	Coll. Ep. $\downarrow$	Coll. Rate $\downarrow$	On-Sphere $\uparrow$	Sphere Err. (cm) $\downarrow$
Vanilla	37.0%	82.0%	54.0%	16.4%	84.3%	2.78
Post-Projection	25.0%	29.0%	14.5%	4.7%	56.4%	4.84
OmniGuide [37]	39.0%	81.0%	52.0%	15.98%	84.9%	2.71
RAIL [4]	37.5%	37.5%	0.0%	0.00%	82.9%	2.73
NUDGE, $\rho_k = 0.01$	42.0%	73.0%	47.0%	13.8%	80.5%	3.00
NUDGE, $\rho_k = 0.04$	<u>66.5%</u>	73.0%	10.0%	0.91%	83.6%	2.86
NUDGE, $\rho_k = 0.07$	71.5%	73.5%	2.5%	0.21%	83.0%	2.87

Note: *Success* is the integrated metric: an episode succeeds only when the robot reaches the goal *and* is collision-free throughout. The remaining columns decompose performance into three groups that a useful safe controller must balance: *Goal* (Reach, the fraction of episodes that reach the goal regardless of collisions), *Collision* (Coll. Ep., the fraction of episodes with any collision; Coll. Rate, the fraction of control steps across all episodes during which the robot is in collision), and *Manifold* (On-Sphere, the percentage of trajectory steps within 5 cm of the sphere surface; Sphere Err., the mean distance from the sphere surface across all steps).

the three ρ_k settings in Table II, fixed across denoising steps, with replanning every eight control steps.

All methods in Table II share the pretrained policy. Reach requires joint distance to the goal below 0.3 rad. Success additionally requires no collision. Coll. Ep. and Coll. Rate measures the percentages of episodes and steps with contact. On-Sphere measures steps within 5 cm of the manifold; Sphere Err. is the mean distance to it.

NUDGE achieves the highest overall success rate by balancing manifold preservation and obstacle avoidance, where each baseline collapses on one or more axes. In both *Post-Projection* and *RAIL* [4], the correction is decoupled from the policy’s score and can disagree with the policy’s intended direction, causing the robot to oscillate and sometimes settle in a local minimum. NUDGE applies the gradient inside the policy’s own denoising loop, so the correction stays aligned with the learned task distribution while pushing the chunk away from collision.

C. Collision Avoidance in LIBERO and Latency

We next test whether NUDGE generalizes when the task constraint is implicit and not directly measurable, the typical setting for learned manipulation. We use the fine-tuned $\pi_{0.5}$ [3] checkpoint, a flow-matching VLA with EEF-delta actions. LIBERO-Object [42] comprises 10 pick-and-place tasks with obstacle-free demonstrations. The fine-tuned $\pi_{0.5}$ achieves 98.4% obstacle-free success. Action chunks are $H = 10$ EEF deltas sampled with the flow-matching Euler integrator over $K = 10$ steps. We inject a spherical obstacle between the object and the basket, lifted 18 cm, with radius 6–8 cm, ± 1.5 cm horizontal and ± 1 cm vertical jitter, and one of six colors. The three-obstacle condition retains this sphere and adds two 6–8 cm spheres along the robot’s critical path (Fig. 4). Both unguided $\pi_{0.5}$, OmniGuide, and NUDGE are evaluated on 100 episodes per method and condition, with ten initial states per task and identical obstacle draws across methods.

All three methods use the same $\pi_{0.5}$ checkpoint; NUDGE and OmniGuide additionally share the EEF-action decoder.

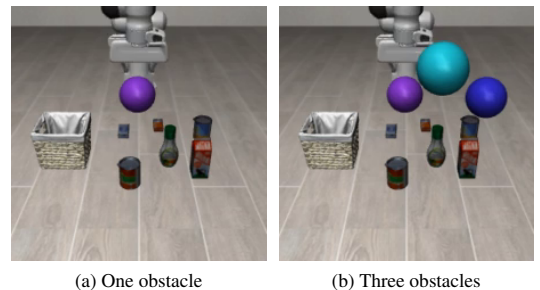


Fig. 4. One- and three-obstacle LIBERO-Object scenes with matched initial robot/object poses and camera view. We note that the additional obstacles may make a collision-free grasp infeasible.

NUDGE constructs a cuRoboV2 ESDF from a single metric-depth view with 2 cm voxels. Simulator masks exclude the robot, target, basket, and table. Its robot representation comprises 41 Panda spheres (Fig. 2) and a 7 cm payload sphere at a fixed offset from `panda_hand`. We set $\rho_k = 0.85$ (Eq. (5)) across all ten tasks. OmniGuide uses VGGT [43] to reconstruct geometry from front, overhead, and agent RGB views, with object-aware CLIP for semantic target filtering. The collision geometries are five points near the end-effector (Fig. 2). Following a hyperparameter search, we use a guidance scale of 0.4 and EEF/wrist thresholds of 0.2/0.1 m. For RAIL, the released implementation does not provide a perception pipeline for constructing its geometric representation from our LIBERO observations. We therefore use simulator ground-truth geometry for collision checking and backup planning.

In RAIL, 113 of the 200 LIBERO episodes terminate after exhausting the backup-search budget, and another 76 reach the episode time limit. Backup control accounts for 76.3% and 83.1% of executed steps in the one- and three-obstacle conditions, respectively. The position-based recovery objective does not encode the complete grasp-and-place sequence. This suggests that frequent intervention can limit task progress even when robot-obstacle contact is avoided.

TABLE III
LIBERO-OBJECT RESULTS

Task	Unguided		OmniGuide		RAIL		NUDGE	
	SR $\uparrow$	CR $\downarrow$	SR $\uparrow$	CR $\downarrow$	SR $\uparrow$	CR $\downarrow$	SR $\uparrow$	CR $\downarrow$
<i>One obstacle</i>								
Can	50	5.70	60	15.19	10	0.00	100	0.00
Cheese	40	6.56	40	17.25	0	0.00	60	9.71
Dressing	20	4.55	40	0.09	0	0.00	40	0.00
BBQ	0	4.72	10	2.22	0	0.00	10	0.03
Ketchup	20	11.78	0	10.55	0	0.00	0	0.10
Tomato	0	31.24	0	28.21	0	0.00	0	10.66
Butter	90	6.14	80	5.89	0	0.00	90	0.00
Milk	0	23.14	0	19.86	0	0.00	30	2.03
Pudding	70	28.58	70	25.37	0	0.00	80	0.50
Juice	10	9.05	0	0.59	0	0.00	30	0.00
Mean	30	13.15	30	12.52	1	0.00	44	2.30
<i>Three obstacles</i>								
Can	10	25.61	10	23.22	0	0.00	20	4.62
Cheese	0	21.31	10	17.43	0	0.00	40	23.56
Dressing	10	27.16	0	40.97	0	0.00	0	15.45
BBQ	0	40.41	10	31.57	0	0.00	0	2.76
Ketchup	0	32.48	0	31.45	0	0.00	0	6.86
Tomato	0	49.41	0	44.17	0	0.00	0	24.90
Butter	10	31.26	30	15.15	0	0.00	40	1.83
Milk	0	42.48	0	37.28	0	0.00	0	10.90
Pudding	10	45.22	10	33.93	0	0.00	30	12.26
Juice	0	67.28	0	60.76	0	0.00	0	36.93
Mean	4	38.26	7	33.59	0	0.00	13	14.01

Note: SR denotes task success rate. CR denotes the percentage of recorded control steps with robot-obstacle contact. All values are percentages.

We measure the environment-representation latency of NUDGE and OmniGuide on 25 paired frames from five LIBERO rollouts. Each pipeline runs independently on an RTX 4090 after five warmup requests. Timing includes input preprocessing, representation construction, and the service round trip, while excluding rendering, disk I/O, policy inference, and action execution. Mean latency is 907.44 ms for OmniGuide and 3.32 ms for NUDGE, a ratio of approximately **273.3 $\times$ faster**. This enables NUDGE to deliver real-time, reactive performance in the real world, which we will introduce next.

D. Real-Robot Evaluation and System Latency

We deploy a fine-tuned π_0 [2] on a UR5. In each trial, the robot picks one of six cubes from a table and places it in a container. An experimenter continuously moves one foam obstacle through the corridor between the cubes and container, challenging both reaching and transport. We use an RTX 5090, CUDA 12.8, PyTorch 2.11, and π_0 in bf16. A cuRoboV2 [33] node fuses live depth from a single RGB-D camera into a 2 cm voxel map. Joint state, base RGB, and wrist RGB arrive over ZMQ. The base policy π_0 is fine-tuned on language-conditioned pick-and-place, with chunk size $H = 50$ and $K = 10$ flow-matching steps. SDF

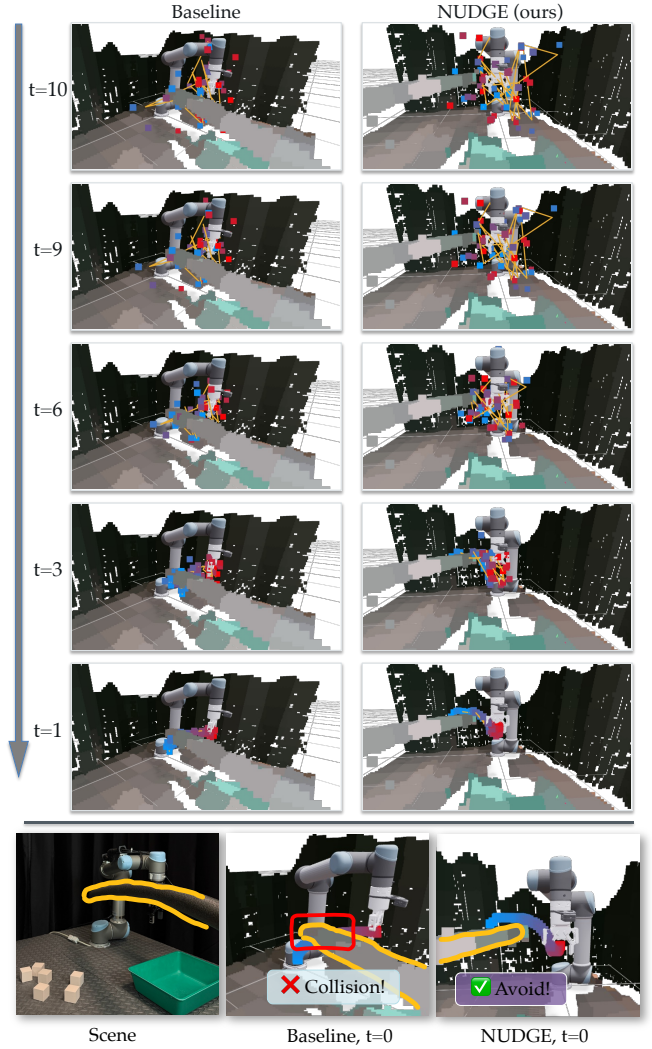


Fig. 5. **Top:** Denoising visualization for a single action chunk. The policy predicts a 50-step chunk of joint deltas. The actions are mapped to end-effector pose and rendered in a color spectrum across denoising steps, from random noise at $t = 10$ down to the final denoised chunk at $t = 0$. The environment is shown as a voxel map. **Bottom, left:** Real-world setup; the obstacle is a pool noodle. **Bottom, middle and right:** Final action chunk at $t = 0$ for the unguided baseline and NUDGE, respectively. The baseline is unaware of the obstacle and collides with it. NUDGE pushes the chunk away from the obstacle.

guidance is injected only at the last two denoising steps. This late-step guidance is also used in image generation [22].

Across 82 consecutive trials, NUDGE completes 62 (75.6%) without obstacle or container contact. These six trials account for the 7.3% failures attributed to NUDGE. Of the remaining 14 failures, 11 concern grasping or placement and three are camera/gripper connection losses. In comparison, across 18 trials, unguided π_0 completes three (16.7%), collides in 13 (72.2%), and has two grasping failures. NUDGE achieves **4.5 times higher completion rate** than baseline. The 82-trial outcome decomposition appears in Fig. 6. Figure 5 visualizes the guided denoising process. On the RTX 5090, Table IV compares π_0 with and without NUDGE in the VLASH framework [45]. Guidance adds an

TABLE IV
UR5 INFERENCE LATENCY (MS)

Stage	π_0	+ NUDGE	Δ
Observation I/O and transfer	5.06	5.54	+0.48
Vision/language encoding	8.94	9.01	+0.07
Causal prefill	17.02	17.48	+0.46
Denoising loop (10 steps)	72.40	76.38	+3.98
Action-expert step, mean	7.13	7.20	+0.07
Guidance (last 2 steps)	—	3.76	+3.76
Post-processing	0.16	0.20	+0.04
Total, eager	104.76	109.90	+5.14
Total, compiled	34.41	36.87	+2.46

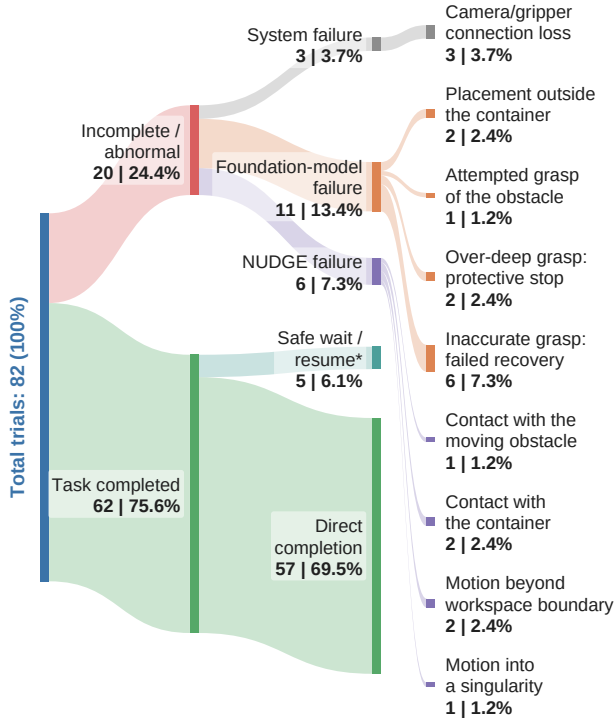


Fig. 6. **Real UR5 outcome breakdown** of all 82 consecutive. In each trial, the robot picked a cube on the table and placed it into a container while a foam obstacle moved continuously through the pick-and-place workspace. NUDGE success rate was 76.5%. The foundation model, rather than NUDGE, was the primary bottleneck, accounting for 11 of the 19 incomplete trials. (*) denotes autonomous hold-and-resume during temporary blockage, without task assistance.

FK pass, an ESDF query, and a backward gradient pass at each of the two enabled denoising steps, totaling only 2.46 ms of end-to-end overhead under `torch.compile`. Each row of the table is the median over 30 timed iterations after 5 warmup runs. Per-stage timings use eager PyTorch with CUDA synchronization. Compiled totals use `torch.compile`, whose CUDA graphs do not expose per-stage timers.

V. DISCUSSION

We presented NUDGE, an inference-time procedure that adds obstacle avoidance to pretrained robot policies based

on diffusion or flow matching by injecting signed-distance-field gradients into the denoising loop. NUDGE preserves task-manifold adherence and achieves 41% collision-free completion versus 16% for the state-of-the-art baseline on 100 matched single-obstacle LIBERO episodes. On a UR5, it achieves 75.6% completion over 82 consecutive trials with a moving obstacle. Guidance adds 2.46 ms per action chunk on the UR5. Natural next steps are extending guidance to articulated and deformable obstacles, modeling held-object geometry more faithfully. Future work will also focus on providing a formal safety guarantee for robot policies.

REFERENCES

- [1] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [2] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *Robotics: Science and Systems*, 2025.
- [3] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, *et al.*, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” in *9th Annual Conference on Robot Learning*, 2025.
- [4] W. Jung, D. Anthony, U. A. Mishra, N. R. Arachchige, M. Bronars, D. Xu, and S. Kousik, “Rail: Reachability-aided imitation learning for safe policy execution,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 3582–3589.
- [5] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 9902–9915. [Online]. Available: <https://proceedings.mlr.press/v162/janner22a.html>
- [6] J. Carvalho, A. T. Le, P. Kicki, D. Koert, and J. Peters, “Motion planning diffusion: Learning and adapting robot motion planning with diffusion models,” *IEEE Transactions on Robotics*, vol. 41, pp. 4881–4901, 2025.
- [7] K. Saha, V. Mandadi, J. Reddy, A. Srikanth, A. Agarwal, B. Sen, A. Singh, and M. Krishna, “Edmp: Ensemble-of-costs-guided diffusion for motion planning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 10 351–10 358.
- [8] M. Seo, Y. Cho, Y. Sung, P. Stone, Y. Zhu, and B. Kim, “Presto: Fast motion planning using diffusion models based on key-configuration environment representation,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 10 861–10 867.
- [9] J. Yang, J. J. Liu, Y. Li, Y. Khaky, K. Shaw, and D. Pathak, “Deep reactive policy: Learning reactive manipulator motion planning for dynamic environments,” in *Proceedings of The 9th Conference on Robot Learning*, vol. 305, 2025, pp. 5056–5071.
- [10] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus, “Safediffuser: Safe planning with diffusion probabilistic models,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [11] J. Zhang, L. Zhao, A. Papachristodoulou, and J. Umenberger, “Constrained diffusers for safe planning and control,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- [12] R. Römer, A. v. Rohr, and A. Schoellig, “Diffusion predictive control with constraints,” in *Proceedings of the 7th Annual Learning for Dynamics & Control Conference*. PMLR, 2025, pp. 791–803.
- [13] J. Yang, S. Jang, and S. Han, “Safeflowmatcher: Safe and fast planning using flow matching with control barrier functions,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=refcXHU1Nh>
- [14] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023.
- [15] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “Rdt-1b: a diffusion foundation model for bimanual manipulation,” in *International Conference on Learning Representations*, 2025.

- [16] S. Jiang, X. Fang, N. Roy, T. Lozano-Pérez, L. P. Kaelbling, and S. Ancha, "Streaming flow policy: Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 27–30 Sep 2025, pp. 238–257. [Online]. Available: <https://proceedings.mlr.press/v305/jiang25a.html>
- [17] J. Jia, G. Li, X. Chen, T. An, Y. Hu, J. Li, X. Guo, and J. Yang, "Action-to-action flow matching," *arXiv preprint arXiv:2602.07322*, 2026.
- [18] D. Gao, B. Zhao, A. Lee, I. Chuang, H. Zhou, H. Wang, Z. Zhao, J. Zhang, and I. Soltani, "VITA: Vision-to-action flow matching policy," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=BTe5VLBJPg>
- [19] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 2679–2713. [Online]. Available: <https://proceedings.mlr.press/v270/kim25c.html>
- [20] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al., "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [21] P. Dhariwal and A. Nichol, "Diffusion models beat gans on image synthesis," *Advances in Neural Information Processing Systems*, vol. 34, pp. 8780–8794, 2021.
- [22] A. Bansal, H.-M. Chu, A. Schwarzschild, S. Sengupta, M. Goldblum, J. Geiping, and T. Goldstein, "Universal guidance for diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2023, pp. 843–852.
- [23] H. Chung, J. Kim, M. T. Mccann, M. L. Klasky, and J. C. Ye, "Diffusion posterior sampling for general noisy inverse problems," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=OnD9zGAGT0k>
- [24] Y. Wang, J. Yu, and J. Zhang, "Zero-shot image restoration using denoising diffusion null-space model," *The Eleventh International Conference on Learning Representations*, 2023.
- [25] M. Du and S. Song, "Dynaguide: Steering diffusion policies with active dynamic guidance," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. [Online]. Available: <https://openreview.net/forum?id=XOW7Yf8qN3>
- [26] S. Liu, I. S. Singh, Y. Xu, J. Duan, and R. Krishna, "Vls: Steering pretrained robot policies via vision-language models," *arXiv preprint arXiv:2602.03973*, 2026.
- [27] M. Park, K. Kim, J. Hyung, H. Jang, H. Jin, J. Yun, H. Lee, and J. Choo, "Acg: Action coherence guidance for flow-based vla models," *arXiv preprint arXiv:2510.22201*, 2025.
- [28] H. Li, Q. Feng, Z. Zheng, J. Feng, Z. Chen, and A. Knoll, "Language-guided object-centric diffusion policy for generalizable and collision-aware manipulation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 12 834–12 841.
- [29] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *2019 18th European Control Conference (ECC)*. Ieee, 2019, pp. 3420–3431.
- [30] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-jacobi reachability: A brief overview and recent advances," in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. IEEE, 2017, pp. 2242–2253.
- [31] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 6023–6029.
- [32] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *International Conference on Machine Learning*. Pmlr, 2017, pp. 22–31.
- [33] B. Sundaralingam, A. Murali, and S. Birchfield, "curobo2: Dynamics-aware motion generation with depth-fused distance fields for high-dof robots," *arXiv preprint arXiv:2603.05493*, 2026.
- [34] A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, and R. Siegwart, "nvblox: Gpu-accelerated incremental signed distance field mapping," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 2698–2705.
- [35] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. D. Ratliff, D. Fox, F. Ramos, and B. Boots, "Storm: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Conference on Robot Learning*. PMLR, 2022, pp. 750–759.
- [36] M. Dalal, J. Yang, R. Mendonca, Y. Khaky, R. Salakhutdinov, and D. Pathak, "Neural mp: A neural motion planner," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 1–8.
- [37] Y. Song, L. Le, Y.-H. Park, J. Wang, J. Shi, L. Liu, J. Gu, E. Eaton, D. Jayaraman, and K. Daniilidis, "Omniguide: Universal guidance fields for enhancing generalist robot policies," *arXiv preprint arXiv:2603.10052*, 2026.
- [38] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.
- [39] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=St1giarCHLP>
- [40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "Chomp: Gradient optimization techniques for efficient motion planning," in *2009 IEEE international Conference on Robotics and Automation*. IEEE, 2009, pp. 489–494.
- [41] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, et al., "Curobo: Parallelized collision-free robot motion generation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 8112–8119.
- [42] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.
- [43] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "Vggt: Visual geometry grounded transformer," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 5294–5306.
- [44] P. Holmes, S. Kousik, B. Zhang, D. Raz, C. Barbalata, M. Johnson-Roberson, and R. Vasudevan, "Reachable sets for safe, real-time manipulator trajectory design," *Robotics: Science and Systems*, 2020.
- [45] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han, "Vlash: Real-time vlas via future-state-aware asynchronous inference," *arXiv preprint arXiv:2512.01031*, 2025.